Week 17 (Days 113-119): RAG, quantisation and inference serving

Status: written in full (Phase 4). This week covers RAG, quantisation, Rust int8, KV cache, batching and speculative decoding (SYLLABUS titles match). llama.cpp remains UNTESTED (no g++/cmake). Every number below was printed by a script in `code/week-17/` on this box (torch 2.15.0+cpu, transformers 5.21.1, Rust via cargo + maturin 1.15.0, 8 CPU threads, no GPU), with logs in `code/week-17/out/*.log`. Scripts were run one at a time.

Week goals

- Embed, chunk and search a real document collection (these course notes), and measure retrieval quality honestly.
- Combine lexical and dense retrieval, add a cross-encoder reranker, and see what each buys.
- Build retrieval-augmented generation with a 135M model and separate retrieval errors from generation errors.
- Implement int8/int4 quantisation from scratch and measure its effect on SmolLM2-135M perplexity.
- Implement simplified GPTQ and AWQ, and write an int8 matmul kernel in Rust with PyO3.
- Write a Llama forward pass with your own KV cache; measure cache speed-up, batching throughput and speculative decoding.
- Weekly project: a RAG system over the notes with fp32, int8 and int4 generators.

```
flowchart LR
  A[Day 113 embeddings + vector search] --> B[Day 114 hybrid + rerank]
  B --> C[Day 115 RAG generation]
  C --> D[Day 116 int8/int4 RTN]
  D --> E[Day 117 GPTQ, AWQ, Rust kernel]
  E --> F[Day 118 KV cache, batching, speculative]
  F --> G[Day 119 project + quiz]
```

Compute note (CPU vs GPU). Everything here fits on a CPU because the models are tiny: the MiniLM embedding model, a MiniLM cross-encoder and SmolLM2-135M. Quantisation is *simulated*: weights are rounded to int8/int4 and then stored dequantised in fp32, so it measures quality loss but not speed. Real speed-ups need kernels that compute on low-bit weights (on GPUs: GPTQ/AWQ/Marlin-style kernels; on CPUs: llama.cpp's). On a GPU you would also see the KV cache and batching effects much more strongly, and serving engines such as vLLM only make sense there.

llama.cpp on this box: not built. This box has `gcc` and `cc` but **no `g++`, no `cmake` and no `make`**, and llama.cpp is C/C++ built with CMake, so it was not built or run here. Its description below comes only from its README (copy saved as `code/week-17/out/docs_llama_cpp_README.md`). vLLM is described from its README (`out/docs_vllm_README.md`); it was not installed (it targets GPUs).

Models and data. From the local Hugging Face cache: `sentence-transformers/all-MiniLM-L6-v2` (verified in earlier weeks), `HuggingFaceTB/SmolLM2-135M` and `-Instruct` (Week 15), and `cross-encoder/ms-marco-MiniLM-L6-v2`, downloaded this week from its official Hub repository (6 files, including `model.safetensors`). The corpus is `weeks/week-01.md` to `week-16.md`: 112 days with content. The retrieval eval set is the 58 interview-style questions found in those notes; each question's own section is **removed** from the index, and the "gold" answer is any chunk from the same day. The RAG QA set (`code/week-17/_qa.py`) has 12 hand-written questions whose exact answer strings appear in weeks 15-16 (the script checks this: none missing).

Day 113: Embeddings, chunking and vector search

Review today: Day 112 (+1), Day 110 (+3), Day 106 (+7), Day 92 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Why retrieval? A language model only knows what is in its weights, frozen at training time, and small models know little. Retrieval-augmented generation (RAG) fetches relevant text at query time and puts it in the prompt. It lets a model use private, recent or large documents without retraining, and it makes answers checkable against a source.

Sentence embeddings. An embedding model maps a text to a vector so that texts with similar meaning are close. MiniLM-L6 produces 384-dimensional vectors: run the encoder, average the token vectors weighted by the attention mask (*mean pooling*),

and L2-normalise. With unit vectors, cosine similarity is just a dot product. These models are trained contrastively on (query, relevant passage) pairs, so "close" means "likely relevant", not "same words".

Chunking. Documents are too long to embed as one vector (MiniLM is used with at most 256 tokens here, and longer input is truncated), and one vector for a whole document blurs its topics. So we split into *chunks*. Trade-offs: - Small chunks are precise but may lose context ("it" without the noun). - Large chunks keep context but dilute the vector and cost more prompt tokens; beyond the model's limit they are silently truncated. - *Overlap* between consecutive windows avoids cutting an idea in half. - *Structure-aware* chunking (by heading or section) follows the author's own boundaries. Prepending the document title ("Day 108 DPO from scratch: ...") to each chunk is a cheap way to keep context.

Vector search. With N chunks and d dimensions, exact (brute-force) search is a matrix product: N.d multiply-adds per query. For a few thousand chunks that is microseconds. Approximate nearest-neighbour indexes (HNSW graphs, IVF clustering, product quantisation) matter at millions of vectors, trading a little recall for speed and memory.

Evaluating retrieval. Recall@k: is a relevant chunk among the top k? MRR (mean reciprocal rank): average of 1/rank of the first relevant chunk. You need queries with known relevant documents; here they come from the notes' own interview questions.

Block 2: Worked example (45 min)

`code/week-17/day113_embed_search.py` parses the notes into 112 days, extracts 58 queries and compares four chunkings. Example query: "Why not train ten separate yes/no classifiers instead of one softmax model?" -> gold day 34.

Verified (out/day113_embed_search.log):

Chunking	Chunks	Embed time	recall@1	recall@5	MRR
64 words, overlap 16	2,012	18.3 s	0.43	0.62	0.52
128 words, overlap 32	1,037	14.5 s	0.40	0.64	0.51
256 words, overlap 64	550	4.7 s	0.34	0.64	0.50
by section	1,099	9.4 s	0.40	**0.71**	**0.54**

- Searching all 58 queries took 0.4-1.5 ms in total (brute force).
- The 128-word index is 1.59 MB in fp32 and 0.40 MB as int8 (per-vector scale); int8 retrieval gave recall@1 0.40, recall@5 0.62, MRR 0.51, almost the same as fp32.

Reading: section chunking was best at recall@5 (0.71), because the notes' headings are good boundaries. 256-word windows were worst at recall@1 (0.34), partly because many of them exceed MiniLM's 256-token input and are truncated. With 58 queries a difference of 0.03 is about two queries, so only the larger gaps mean anything (Day 110).

Block 3: Lab (2h)

1. Run the script (under a minute) and compare with the log.
2. Remove the "Day N title:" prefix from chunks and re-run. How much does recall drop?
3. Add chunking by paragraph (split on blank lines, merge paragraphs to about 128 words).
4. Print the top 5 chunks for three failed queries. Are they wrong, or relevant but from another day (the gold label is strict)?
5. Implement a tiny IVF index: k-means the chunk vectors into 32 clusters (your Week 8 code), search only the 4 nearest clusters, and measure recall loss and speed.

Block 4: Practice (1h)

1. Why normalise embeddings before a dot product?
2. Why can very large chunks hurt even when the answer is inside them?
3. What is the cost of brute-force search for 1M chunks of dimension 384, per query?
4. Why is "relevant chunk = same day" a strict, imperfect label?

Answers. (1) Then dot product = cosine similarity, independent of vector length. (2) One vector must summarise many topics, and text past the model's limit is truncated (MiniLM 256 tokens here). (3) 384M multiply-adds, about 1.5 GB of fp32 vectors in memory; fine on a GPU, slow per query on a CPU, hence ANN indexes. (4) Other days may also answer the question (topics repeat in reviews), so some "misses" are actually useful.

Block 5: Review (1h)

Self-check: - [ ] I can explain mean pooling and why embeddings are normalised. - [ ] I can list three chunking strategies and their trade-offs. - [ ] I can compute recall@k and MRR by hand for a small example. - [ ] I can read the Day 113 table and say which differences are meaningful.

Extra notes

The eval set is built from the same notes, so it is "in-domain" and the queries use the notes' vocabulary. Real users phrase questions differently; collect real queries when you can. The int8 index result shows that embedding vectors tolerate quantisation well, which is why vector databases routinely compress them.

Going deeper

- Read the Sentence-BERT paper ("Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks") for how bi-encoders are trained.
- Read the HNSW paper ("Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs").
- Read the FAISS documentation's overview of index types.

Interview-style question (15 min)

"How would you choose a chunk size for a RAG system over internal documentation?" Points: build a labelled query set first; test several sizes and structure-aware chunking; respect the embedding model's token limit; include titles or headers in chunks; consider the generator's context budget (k x chunk size); measure recall@k and end-to-end answer quality, not one of them alone.

Common mistakes

- Exceeding the embedding model's maximum length without noticing truncation.
- Mixing embeddings from different models, or forgetting to normalise.
- Evaluating retrieval on two or three hand-picked queries.

What to do next

Tomorrow: BM25, hybrid retrieval and a cross-encoder reranker on the same 58 queries.

Day 114: Hybrid retrieval, reranking and retrieval evaluation

Review today: Day 113 (+1), Day 111 (+3), Day 107 (+7), Day 93 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Lexical retrieval: BM25. BM25 scores a document by the query terms it contains, weighted by inverse document frequency (rare words count more), with term frequency saturating (parameter k1) and a length normalisation (parameter b). It needs no training, is fast, and is very good at exact names, numbers, code identifiers and rare terms, which dense models can blur.

Dense retrieval (Day 113) is good at paraphrase and meaning, worse at exact rare tokens. The two make *different* errors, which is why combining them helps.

Hybrid by reciprocal rank fusion (RRF). Score each document by Sum over retrievers of 1/(k + rank), with k ~= 60. RRF uses only ranks, so you do not need to calibrate BM25 scores against cosine similarities.

Reranking. A *bi-encoder* embeds query and document separately, so documents can be pre-computed. A *cross-encoder* reads query and document *together* through one transformer and outputs a relevance score. It is much more accurate, because every query token can attend to every document token, but it must run once per (query, candidate) pair, so it is only used on the top 20-100 candidates from a cheaper first stage. This two-stage pattern (recall-oriented retrieval, then precision-oriented reranking) is standard.

Evaluation discipline. Keep the query set fixed, report recall@k and MRR at the k you will actually put in the prompt, report latency per query, and look at failures by hand.

Block 2: Worked example (45 min)

`code/week-17/day114_hybrid_rerank.py` uses the 128/32 index from Day 113 (1,037 chunks).

Verified (out/day114_hybrid_rerank.log):

Retriever	recall@1	recall@5	MRR	Cost
Dense (MiniLM)	0.40	0.64	0.51	
BM25	0.41	0.67	0.51	index + scoring all 58 queries 0.1 s
Hybrid RRF	0.41	0.76	0.56	
Dense top-20 + cross-encoder	**0.50**	**0.81**	**0.63**	200 ms/query
Hybrid top-20 + cross-encoder	0.48	0.79	0.62	213 ms/query

Reading: on these notes, simple BM25 was as good as dense retrieval (0.67 vs 0.64 recall@5). Fusion raised recall@5 to 0.76 because the two retrievers miss different queries. The cross-encoder added the most: recall@1 0.40 -> 0.50. After reranking, dense and hybrid first stages ended within one or two queries of each other (0.81 vs 0.79), which is noise at n = 58. The price is latency: about 200 ms per query on this CPU for 20 pairs.

Block 3: Lab (2h)

1. Run the script (about 30 seconds).
2. Tune BM25's k1 in {0.9, 1.5, 2.0} and b in {0.4, 0.75, 1.0}. Keep a separate tuning subset of queries to avoid tuning on the test set.
3. Rerank the top 50 instead of the top 20. Does recall@5 improve, and what happens to latency?
4. Replace RRF with a weighted sum of min-max-normalised scores. Compare.
5. Find two queries where BM25 wins and dense loses, and explain why from the words involved.

Block 4: Practice (1h)

1. Why does RRF not need score calibration?
2. Why can't a cross-encoder be used to search all 1,037 chunks for every query in a large system?
3. Compute the RRF score (k = 60) of a chunk ranked 1st by BM25 and 10th by dense.
4. Why might BM25 do well on these particular notes?

Answers. (1) It uses only ranks, which are comparable across retrievers. (2) It needs one forward pass per (query, chunk) pair; at about 10 ms per pair here (200 ms for 20), 1,037 chunks would take about 10 s per query. (3) 1/61 + 1/70 ~= 0.0164 + 0.0143 = 0.0307. (4) Questions and notes share technical vocabulary (DPO, LoRA, perplexity), and rare technical terms are what BM25 rewards.

Block 5: Review (1h)

Self-check: - [ ] I can explain BM25's idf, k1 and b. - [ ] I can implement RRF. - [ ] I can explain bi-encoder vs cross-encoder and why reranking is two-stage. - [ ] I can read the Day 114 table and name the biggest and the insignificant gains.

Extra notes

The same reranker model was trained on web search data (MS MARCO), not course notes; it still transferred well here. Retrieval quality is usually the main limit on RAG quality: tomorrow shows a 135M generator failing even when the right text is retrieved, which is the other limit.

Going deeper

- Read the BM25 chapter of "Introduction to Information Retrieval" (Manning, Raghavan, Schütze), freely available from the authors' site.
- Read the RRF paper (Cormack, Clarke, Büttcher, 2009).
- Look up ColBERT for a "late interaction" design between bi- and cross-encoders.

Interview-style question (15 min)

"Your RAG system misses answers that contain product codes like 'XK-4402'. What would you change?" Points: dense embeddings blur rare identifiers; add BM25 or keyword search and fuse; check tokenisation and normalisation of codes; add metadata filters; evaluate on a query set containing such codes.

Common mistakes

- Tuning retrieval hyper-parameters on the test queries.
- Reporting recall@20 when the prompt only gets the top 3.
- Ignoring reranker latency in the serving budget.

What to do next

Tomorrow: put the retrieved chunks in a prompt and see whether a 135M model can use them.

Day 115: RAG generation and grounding evaluation

Review today: Day 114 (+1), Day 112 (+3), Day 108 (+7), Day 94 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

The RAG prompt. Retrieve the top k chunks, put them in the prompt with clear separators and an instruction ("Answer using only the context. If the answer is not there, say you don't know."), then ask the question. Design choices: k (more context raises recall but costs tokens and can distract a small model), the order of chunks, whether to cite chunk ids, and the chat template (Week 15).

Two failure modes. A wrong RAG answer is either a *retrieval* failure (the needed text was not in the prompt) or a *generation* failure (it was there, but the model ignored, misread or contradicted it). Measure them separately: - *Context recall*: fraction of questions whose answer string appears in the retrieved context. - *Answer accuracy*: fraction of answers containing the gold answer. - *Faithfulness / grounding*: whether claims in the answer are supported by the context. With short factual answers, a string check is a crude proxy; LLM judges and human checks are more general (Day 110 warns about judge biases).

Baselines. Always compare with *closed-book* (no context: what does the model already know?) and an *oracle* (give the passage known to contain the answer: what does perfect retrieval buy?). The gap closed-book -> RAG is the value of retrieval; the gap RAG -> oracle is retrieval's remaining error; oracle's own score is the generator's ceiling.

Block 2: Worked example (45 min)

`code/week-17/day115_rag_generate.py` uses the hybrid retriever with reranking, top 3 chunks, SmolLM2-135M-Instruct, greedy decoding, and 12 questions whose answers are numbers or short phrases from Weeks 15-16 (all 12 answer strings were confirmed present in the notes).

Verified (out/day115_rag_generate.log): - Context recall@3: **0.92** (11 of 12 questions had the answer in the context). - Accuracy: closed-book **0.00**, RAG **0.25**, "oracle" passage **0.08**. - Mean RAG prompt length: 816 tokens. Whole run: 30 s. - Typical failures with the right context present: an answer of "192 perplexity" for a different value, and "2 trainable parameters" for a question whose answer in the context was 2,442,817.

Reading: retrieval worked (0.92) but the 135M generator could not reliably copy facts from an 816-token prompt. The "oracle" scored *below* RAG because our oracle is crude: the first chunk containing the answer string, which may mention the number in passing without the question's context. So "the model ignored the context" is the main error here, and a bigger generator is the fix; with this model, better retrieval cannot raise accuracy much above 0.25. Twelve questions is a tiny set: one question is 0.08.

Block 3: Lab (2h)

1. Run the script (about 30 s). Read all 12 answers next to their contexts.
2. Try k = 1 and k = 5. Does a shorter prompt help this small model?
3. Change the instruction to "Copy the exact number from the context". Measure again.
4. Improve the oracle: choose the chunk that contains the answer and has the highest BM25 score for the question.

5. Write a simple grounding check: for numeric answers, is every number in the answer present in the context?

Block 4: Practice (1h)

1. Context recall is 0.92 but accuracy 0.25. Where is the main error?
2. Why is closed-book 0.00 not surprising here?
3. Why can a crude oracle score below RAG?
4. Name one risk of using an LLM judge for faithfulness.

Answers. (1) Generation: the model had the answer in 11 of 12 prompts and got 3 right. (2) The questions are about numbers from our own logs, which no model could have memorised. (3) The oracle chunk contains the answer string but may lack the context that links it to the question, while reranked chunks are relevant to the question. (4) Judges have position, length and self-preference biases (Day 110) and can themselves miss unsupported claims.

Block 5: Review (1h)

Self-check: - [ ] I can design a RAG prompt and explain the choice of k. - [ ] I can separate retrieval failures from generation failures with context recall. - [ ] I can explain why closed-book and oracle baselines matter. - [ ] I can state what Day 115's numbers do and do not show.

Extra notes

With a 135M model, RAG is mainly a teaching tool; the pipeline is the same as with larger models, which follow context far better. Keep the metrics honest: a string match counts "2,442,817" vs "2442817" as different unless you normalise.

Going deeper

- Read the original RAG paper (Lewis et al., 2020, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks").
- Read "Lost in the Middle: How Language Models Use Long Contexts" (Liu et al.) on position effects in long prompts.

Interview-style question (15 min)

"Your RAG chatbot gives wrong answers. How do you find out why?" Points: build a labelled set; measure context recall to split retrieval vs generation errors; inspect failures; compare closed-book and oracle; then fix retrieval (chunking, hybrid, reranking) or generation (prompt, model size, fewer chunks), and re-measure.

Common mistakes

- Judging RAG by a few demo questions.
- Blaming the retriever without measuring context recall.
- Not normalising answers before string matching.

What to do next

Tomorrow: making the generator smaller in memory with int8 and int4 quantisation.

Day 116: Quantisation from scratch (RTN int8/int4)

Review today: Day 115 (+1), Day 113 (+3), Day 109 (+7), Day 95 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Why quantise? LLM inference at small batch is limited by memory bandwidth: each generated token reads every weight. Storing weights in 8 or 4 bits instead of 32 or 16 cuts memory and, with suitable kernels, time. The cost is rounding error.

Uniform quantisation. For a group of weights w, choose a scale s and (optionally) a zero point z; store q = clamp(round(w/s) + z, qmin, qmax) and reconstruct ? = s.(q - z). - *Symmetric*: z = 0, s = max|w| / (2^(b-1) - 1). Simple, good for roughly symmetric weights. - *Asymmetric*: uses min and max, so the full integer range covers [min, max]. - *Round-to-nearest (RTN)* is the simplest choice: each weight rounded independently.

Granularity. One scale for a whole matrix (*per-tensor*) is ruined by outliers: one large weight forces a large scale and all small weights round to zero. *Per-channel* (one scale per output row) and *group-wise* (one scale per g consecutive weights, e.g. 64) adapt to local ranges, at the cost of storing more scales. Group size must divide the row length: SmolLM2's 576-wide rows allow 32 or 64 but not 128.

Measuring. Relative weight error ||W - ?||/||W|| is cheap but indirect; what matters is model quality, here perplexity on held-out text (Shakespeare validation, as in Week 15).

Block 2: Worked example (45 min)

`code/week-17/day116_quant_rtn.py` quantises all 210 linear layers of SmolLM2-135M (106.2M weights; the tied embedding / lm_head stays fp32) and evaluates perplexity.

Verified (out/day116_quant_rtn.log): fp32 baseline perplexity **26.29**, linear weights 424.7 MB. Example outlier: layer 0 `down_proj` max|w| = 2.875 vs mean |w| = 0.1441 (20x).

Scheme	Rel. weight error	Size (incl. scales)	Perplexity
int8 per-tensor	0.0364	106.2 MB	28.55
int8 per-channel	0.0085	106.5 MB	**26.50**
int4 per-tensor	0.5832	53.1 MB	1,561,367.93
int4 per-channel	0.1547	53.4 MB	64.84
int4 group-64	0.1128	56.4 MB	36.89
int4 group-32	0.1006	59.7 MB	33.27
int4 group-64 asymmetric	0.0941	59.7 MB	33.69
int3 group-64	0.2623	43.1 MB	231.67

Each evaluation took about 1 s. Reading: int8 per-channel is almost free (26.29 -> 26.50) at a quarter of the memory. At 4 bits granularity decides everything: per-tensor destroys the model, per-channel doubles perplexity, group-32 costs about 7 points. Smaller groups and asymmetric zero points cost about 3 MB more for the scales and zero points. At 3 bits, RTN breaks down. Weight error predicts the order of the rows but not the size of the perplexity change.

Block 3: Lab (2h)

1. Run the script (about 30 s).
2. Plot a histogram of one layer's weights and mark the int4 per-tensor levels. Why do most weights round to 0?
3. Find the five layers with the largest relative error under int4 per-channel. Keep only those in int8 and re-measure perplexity (mixed precision).
4. Quantise activations too (per-token int8) for one layer and measure output error.
5. Compute exact storage for int4 group-64 with fp16 scales and compare with the logged size.

Block 4: Practice (1h)

1. Quantise w = [0.1, -0.4, 0.25, 2.0] to symmetric int4 per-tensor (qmax = 7). What is the scale and what is reconstructed?
2. Why does per-tensor int4 fail but per-channel survive?
3. What is the storage overhead of fp16 scales for group size 64 at int4?
4. Why can relative weight error mislead?

Answers. (1) s = 2.0/7 ~= 0.286; q = round([0.35, -1.4, 0.875, 7]) = [0, -1, 1, 7]; ? ~= [0, -0.286, 0.286, 2.0]: the small weights are badly represented. (2) Outliers in some rows set a huge global scale; per-row scales isolate them. (3) 16 bits per 64 weights = 0.25 bits per weight, about 6% on top of 4 bits. (4) Errors in some layers (or directions that align with large activations) matter far more for the output than others.

Block 5: Review (1h)

Self-check: - [ ] I can write symmetric and asymmetric quantise/dequantise. - [ ] I can explain per-tensor, per-channel and group-wise scales. - [ ] I can explain why outliers matter. - [ ] I can read the Day 116 table and pick a scheme for a memory budget.

Extra notes

This is simulated quantisation: weights were rounded and stored back in fp32, so nothing got faster; the sizes are what the integer storage would take. A real deployment stores packed integers and uses a kernel that dequantises on the fly (Day 117's Rust kernel is a small example).

Going deeper

- Read "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale" (Dettmers et al.) on outlier features.
- Read the "A Survey of Quantization Methods for Efficient Neural Network Inference" (Gholami et al.).

Interview-style question (15 min)

"You need to run a model in a quarter of its fp16 memory. What would you try?" Points: int4 weights with group-wise scales; measure perplexity and task metrics; use calibration-based methods (GPTQ/AWQ) instead of RTN; keep sensitive layers (embeddings, lm_head, outlier layers) in higher precision; check kernel support on the target hardware; remember the KV cache also needs memory.

Common mistakes

- Using per-tensor scales at 4 bits.
- Reporting weight error instead of model quality.
- Claiming speed-ups from simulated quantisation.

What to do next

Tomorrow: use calibration data to quantise better (GPTQ, AWQ) and write a real int8 kernel in Rust.

Day 117: GPTQ/AWQ concepts, plus a Rust/PyO3 int8 kernel

Review today: Day 116 (+1), Day 114 (+3), Day 110 (+7), Day 96 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

What we really want. Not small weight error but small *output* error: minimise ||XW - X?|| over real inputs X. A few hundred to a few thousand calibration tokens give X.

GPTQ (idea). Quantise the weights of a layer one input-column at a time; after rounding a column, update the *remaining, not yet quantised* columns to compensate for the error, using the inverse of the Hessian H = X^TX (+ damping). Weights that correlate with the rounded one absorb its error. It is a fast approximation of the classical Optimal Brain Surgeon idea, applied layer by layer. In the paper the inputs of each layer are taken from the already-quantised previous layers; our version takes them once from the fp32 model (a simplification).

AWQ (idea). A small fraction of input channels carry large activations, and errors in the weights that multiply them dominate the output. AWQ scales those weight columns *up* by s = (mean |x|)^? before quantising, and divides the input by s (mathematically identity, foldable into the previous layer), so important weights get relatively finer rounding. ? is chosen per layer by grid search on calibration output error. No weight updates, just scales.

Kernels. Low-bit weights only save time if a kernel reads the packed integers and computes with them. On CPU, the key costs are memory traffic and vectorisation; on GPU, dedicated kernels unpack int4 inside the matmul.

Rust and PyO3. PyO3 lets Rust functions be called from Python, maturin builds them into a wheel, the `numpy` crate gives zero-copy access to arrays, and rayon parallelises loops across threads.

Block 2: Worked example (45 min)

`code/week-17/day117_gptq_awq.py`: 2,048 calibration tokens from Shakespeare *training* text (perplexity is on validation text); computing all Hessians took 1.9 s.

Verified (out/day117_gptq_awq.log):

int4 scheme	Method	Output error (calib.)	Perplexity	Time
per-channel	RTN	0.0223	64.84	0.7 s
per-channel	AWQ-style (mean ? 0.27)	0.0161	44.04	1.9 s
per-channel	GPTQ	0.0048	44.31	14.1 s
group-64	RTN	0.0127	36.89	
group-64	AWQ-style (mean ? 0.33)	0.0077	**31.17**	
group-64	GPTQ	0.0026	31.96	13.7 s

(fp32 baseline: 26.29.) Reading: both methods recover much of RTN's loss: per-channel 64.84 -> about 44. GPTQ reaches by far the lowest output error on calibration data, but its perplexity is not better than AWQ's, which suggests it fits the 2,048 calibration tokens too closely (and our one-shot input collection lets layer errors compound unchecked). AWQ is much cheaper.

`code/week-17/rust_qmatmul/` (built with `maturin develop --release` in 8.6 s) and `day117b_rust_int8_matmul.py`, on one 1,536 -> 576 linear layer (576 output rows, one fp32 scale each):

Verified (out/day117b_rust_int8_matmul.log): max difference vs torch on the dequantised weights 2.38e-06 (correct); vs the original fp32 weights 5.72e-02 (quantisation error). Weight memory: 3,538,944 B fp32 vs 884,736 B int8 + 2,304 B scales.

Batch	torch fp32	torch dequant+matmul	Rust 1 thread	Rust rayon 8 threads
1	15.9 µs	982.7 µs	439.4 µs	145.2 µs
64	122.5 µs	1,067.5 µs	26,425.9 µs	4,056.3 µs

Reading: our kernel beats "dequantise then matmul" at batch 1 (145 vs 983 µs) but is about 9x slower than torch's optimised fp32 BLAS (15.9 µs), and far slower at batch 64 because it has no blocking or SIMD tuning. The win is 4x less weight memory, not speed. A lesson in measurement: the first single-process benchmark made torch about 400x slower because rayon's idle threads kept spinning and competed with torch's threads; each timing now runs in its own subprocess.

Block 3: Lab (2h)

1. Run both scripts (GPTQ about 30 s; build plus benchmark under a minute).
2. GPTQ: re-collect each layer's inputs from the partly quantised model and compare perplexity.
3. GPTQ: try 512 and 8,192 calibration tokens. Does the overfitting gap change?
4. AWQ: plot the chosen ? per layer type (q, k, v, o, gate, up, down).
5. Rust: process 4 output rows at once to reuse loads of x, and measure. Then try int4 packing (two weights per byte).

Block 4: Practice (1h)

1. Why does GPTQ need the Hessian X^TX?
2. Why is AWQ's scaling "free" at inference?
3. Why can lower calibration output error fail to give lower perplexity?
4. Why is the Rust kernel slower than torch fp32 at batch 1 despite reading 4x less memory?

Answers. (1) The output error of a weight change ? is ?^T(X^TX)?, so H tells which remaining weights can compensate. (2) The 1/s on the input can be folded into the previous layer's weights or norm. (3) Overfitting to a small calibration set, and per-layer errors interact across layers. (4) At this size the matrix (3.5 MB) fits in cache, so memory is not the bottleneck; BLAS uses SIMD and blocking that our naive loop lacks.

Block 5: Review (1h)

Self-check: - [ ] I can explain GPTQ's error compensation in two sentences. - [ ] I can explain AWQ's activation-aware scaling. - [ ] I can build a PyO3 extension with maturin. - [ ] I can explain the rayon contention problem and how it was fixed.

Extra notes

Our GPTQ and AWQ are simplified teaching versions (no act-order, no clipping search, inputs from fp32). Production libraries implement the full methods plus packed kernels. Treat our numbers as a comparison of ideas on one tiny model, not of the published tools.

Going deeper

- Read "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers" (Frantar et al.).
- Read "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration" (Lin et al.).
- Read the PyO3 user guide and the maturin documentation.

Interview-style question (15 min)

"Compare GPTQ and AWQ." Points: both are post-training and use calibration data; GPTQ updates weights to compensate errors using second-order information; AWQ only rescales channels based on activation magnitude; AWQ is cheaper and less prone to overfitting calibration data; both need dedicated kernels to be fast; measure on your own task.

Common mistakes

- Calibrating and evaluating on the same text.
- Benchmarking two thread pools in one process.
- Comparing a naive kernel with BLAS and calling the difference "quantisation overhead".

What to do next

Tomorrow: the serving side: KV cache, batching and speculative decoding with your own Llama forward pass.

Day 118: Inference serving: KV cache, batching, speculative decoding, vLLM/llama.cpp overview

Review today: Day 117 (+1), Day 115 (+3), Day 111 (+7), Day 97 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Prefill and decode. Generation has two phases: *prefill* processes the whole prompt in parallel (compute-bound), and *decode* produces one token per step (memory-bound at small batch, because each step reads all weights for little compute).

KV cache. In causal attention, the keys and values of past tokens never change. Store them per layer; each decode step then computes Q, K, V only for the new token and attends to the cache, instead of re-running the whole sequence. Cost per step drops from O(t) full forwards' worth to one token's worth. The price is memory: 2 (K and V) x layers x KV heads x head dim x tokens x bytes. Grouped-query attention (GQA, used by SmolLM2) shrinks it by sharing K/V heads across query heads.

Batching. Decoding several sequences together reuses each weight read for several tokens, so total throughput rises until compute becomes the limit; per-sequence speed falls a little. *Continuous batching* lets sequences join and leave the batch at every step instead of waiting for the slowest one. *PagedAttention* (vLLM) stores the KV cache in fixed-size blocks, like virtual-memory pages, to avoid fragmentation and allow sharing prefixes.

Speculative decoding. A cheap *draft* model proposes gamma tokens; the *target* model checks all of them in one forward pass and keeps the longest prefix it agrees with (for greedy: matching argmax), plus one token of its own. Output is identical to the target's; the speed-up depends on the acceptance rate and on how cheap the draft is. A bad draft makes it slower.

Engines (from their READMEs, not run here). - **vLLM** (`out/docs_vllm_README.md`): lists PagedAttention, continuous batching, chunked prefill, prefix caching, quantisation (FP8, INT8, INT4, GPTQ/AWQ, GGUF and others), speculative decoding (n-gram, suffix, EAGLE, DFlash) and an OpenAI-compatible API server; the paper is "Efficient Memory Management for Large Language Model Serving with PagedAttention". It is GPU-oriented and was not installed. - **llama.cpp** (`out/docs_llama_cpp_README.md`): "Plain C/C++ implementation without any dependencies", 1.5- to 8-bit integer quantisation, Apple silicon (Metal) and CUDA support, GGUF models (loadable with `-hf`), and an OpenAI-compatible server (`llama-server`). **Not built here**: the box has no `g++`, `cmake` or `make`. On a machine with a C++ toolchain, follow its README's build instructions.

Block 2: Worked example (45 min)

`code/week-17/_tinyllama.py` implements SmolLM2's forward pass from scratch (RMSNorm, RoPE, GQA, SwiGLU) with a KV cache class; `day118_kv_cache.py` benchmarks it.

Verified (out/day118_kv_cache.log): - Correctness: our logits vs Hugging Face's: max difference **0.00e+00**. - Greedy decoding of 64 tokens: no cache 27.3 tok/s (2.35 s); **our KV cache 67.8 tok/s** (0.94 s), identical tokens; HF `generate` 61.7 tok/s, same tokens. Speed-up from the cache: 2.5x. - KV cache per sequence: 5.9 MB at 128 tokens, 23.6 MB at 512, 94.4 MB at 2,048 (linear in length).

Batch	Total tok/s	Per-sequence tok/s
1	63.6	63.6
4	186.3	46.6
16	559.8	35.0

- Self-speculative decoding (draft = first k layers of the same model with its lm_head, gamma = 4): k = 10 -> acceptance 0.03, 58 target forwards, 24.0 tok/s; k = 20 -> acceptance 0.06, 51 target forwards, 20.1 tok/s. Output equals greedy in both cases.

Reading: the cache gives 2.5x at only 64 tokens; the gain grows with length. Batch 16 gives 8.8x the throughput of batch 1 while each sequence slows to 35.0 tok/s, the classic latency/throughput trade-off. Speculative decoding was *slower* than plain decoding: an early-exit draft of this model almost never predicts the next token (acceptance <= 0.06), so we paid for draft steps and saved few target steps. A trained small draft model or an EAGLE-style head would be needed.

Block 3: Lab (2h)

1. Run the script (about a minute).
2. Measure tok/s with and without the cache at 32, 128 and 256 generated tokens and plot the ratio.
3. Compute the KV cache size from the config formula and check it against the logged 5.9 MB at 128 tokens.
4. Add a simple continuous-batching loop: start a new prompt when one sequence finishes.
5. Speculative decoding: use an n-gram draft (copy the continuation of the last matching bigram in the prompt) on a repetitive prompt, and measure acceptance.

Block 4: Practice (1h)

1. Why is decoding memory-bound at batch 1?
2. If acceptance is 0.06 with gamma = 4, roughly how many tokens does each target forward produce?
3. Why does total throughput rise with batch but per-sequence speed fall?
4. Why couldn't llama.cpp be built on this box?

Answers. (1) Each step reads all weights to produce one token; arithmetic per byte read is tiny. (2) Barely more than 1: on average 0.06 x 4 ~= 0.24 draft tokens are accepted plus the target's own token, consistent with 58 target forwards for 64 tokens at k = 10 (acceptance 0.03). (3) One weight read serves more tokens, but each step does more compute, so it takes longer. (4) It needs a C++ compiler and CMake; only gcc (C) is installed.

Block 5: Review (1h)

Self-check: - [ ] I can explain prefill vs decode and why the KV cache helps. - [ ] I can compute KV cache memory. - [ ] I can explain continuous batching and PagedAttention. - [ ] I can explain when speculative decoding speeds things up and when it does not.

Extra notes

On a GPU every effect is bigger: batching scales much further before compute saturates, which is why serving engines batch aggressively, and KV cache memory, not weights, often limits how many users fit. On this CPU, all measurements are a few seconds and somewhat noisy; repeat them before trusting small differences.

Going deeper

- Read the vLLM PagedAttention paper named above and the blog post linked in the vLLM README.
- Read "Fast Inference from Transformers via Speculative Decoding" (Leviathan et al.).
- Read the llama.cpp README for GGUF and its quantisation types.

Interview-style question (15 min)

"Your LLM service has high latency under load. What do you look at?" Points: prefill vs decode time; batch size and continuous batching; KV cache memory and paging; quantisation of weights (and KV cache); prompt length and prefix caching; speculative decoding if acceptance is high; measure p50/p99 latency and throughput separately.

Common mistakes

- Forgetting RoPE positions when appending to the cache.
- Comparing tok/s across different numbers of generated tokens.
- Assuming speculative decoding always helps.

What to do next

Tomorrow: the weekly project ties retrieval, quantisation and generation together.

Day 119: PROJECT W17: RAG over the notes with a quantised generator; Quiz 17

Review today: Day 118 (+1), Day 116 (+3), Day 112 (+7), Day 98 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Project brief. Build a question-answering system over Weeks 1-16 of these notes: 1. Index: 128-word chunks with 32-word overlap, MiniLM embeddings, plus BM25. 2. Retrieval: hybrid RRF, cross-encoder rerank of the top 20, keep the top 3. 3. Generator: SmolLM2-135M-Instruct in fp32, int8 per-channel, and int4 group-64 (RTN from Day 116). 4. Evaluation: the 12 QA questions (context recall@3, accuracy, agreement with fp32 answers, seconds per answer) and perplexity on Shakespeare validation text. 5. Write a one-page report with the system diagram, the table and honest limits.

What the project tests. Whether you can connect components and measure each one: retrieval quality independent of generation, generation quality given retrieval, and the cost of compression. "Same answers as fp32" is a useful extra metric for quantisation: it measures change in behaviour even when accuracy is too coarse to see it.

Design questions to answer in the report: how much would a bigger generator help (Day 115's diagnosis), which quantisation you would deploy, and what you would change for 100x more documents (ANN index, metadata filters, caching).

Block 2: Worked example (45 min)

`code/week-17/day119_project.py`.

Verified (out/day119_project.log): index of 1,037 chunks built in 10.1 s; retrieval with reranking 250 ms per question; context recall@3 **0.92**.

Generator	Accuracy	Same answer as fp32	s/answer	Perplexity	Linear weights
fp32	0.25	12/12	0.77	32.05	425 MB
int8 per-channel	0.25	9/12	0.78	32.11	106 MB
int4 group-64	0.17	0/12	0.94	43.69	53 MB

(Perplexity here is of the Instruct model, so it differs from Day 116's base model numbers. The int4 size is integer storage without scales.)

Reading: int8 kept accuracy and perplexity (32.05 -> 32.11) but changed 3 of 12 answers, showing that "no perplexity change" does not mean "identical behaviour". int4 group-64 changed every answer and lost one correct answer (0.25 -> 0.17, i.e. 3 -> 2 of 12). Speed did not improve because quantisation is simulated; int4 was even slower per answer (probably because it produced longer answers; not measured). Retrieval was not the bottleneck: generation was.

Block 3: Lab (2h)

1. Run the project (about 1-2 minutes) and check every number against the log.
2. Swap int4 RTN for the AWQ-style group-64 quantisation from Day 117 and re-measure.
3. Log the number of generated tokens per answer to test the explanation for int4's slower answers.
4. Write the report (one page): diagram, table, three findings, three limits.

5. Optional: with a GPU or a C++ toolchain elsewhere, export the model to GGUF and serve it with llama.cpp; compare real int4 speed with this simulation.

Block 4: Practice (1h)

1. Why report "same answers as fp32" in addition to accuracy?
2. Accuracy 0.25 vs 0.17: is that a significant difference?
3. What would you change first to raise accuracy?
4. Why didn't int8 make answers faster?

Answers. (1) It detects behaviour change that 12-question accuracy is too coarse to show. (2) No: it is one question out of 12. (3) The generator (a larger instruction-tuned model), because context recall is already 0.92. (4) The weights were dequantised to fp32; a real int8 kernel would be needed.

Block 5: Review (1h)

Self-check: - [ ] My project reproduces the logged numbers. - [ ] I can explain each component's contribution with a number. - [ ] I can state the limits: 12 questions, a 135M generator, simulated quantisation, notes written by us. - [ ] I completed Quiz 17 below.

Extra notes

Limits, stated plainly: the eval sets are small (58 retrieval queries, 12 QA questions); the corpus and the queries were written by the same author; the generator hallucinates even with correct context; nothing was measured on a GPU; llama.cpp and vLLM were not run.

Going deeper

- Read the RAG paper and the vLLM paper again with the project in mind.
- Look up RAG evaluation frameworks (for example, RAGAS) and compare their metrics with the ones here.

Interview-style question (15 min)

"Walk me through a RAG system you built and how you evaluated it." Use this project: components, context recall 0.92 vs accuracy 0.25, the diagnosis that generation was the bottleneck, the int8/int4 trade-off and the honest limits.

Common mistakes

- Reporting only end-to-end accuracy.
- Quantising and claiming speed without real kernels.
- Drawing conclusions from one-question differences.

What to do next

Week 18 starts with the Month-4 checkpoint exam (Day 120), then reinforcement learning basics.

Weekly quiz: Week 17

1. What does mean pooling do in a sentence-embedding model, and why normalise the result?
2. In Day 113, which chunking gave the best recall@5, and what was it?
3. What is reciprocal rank fusion and why is it convenient?
4. Why is a cross-encoder more accurate than a bi-encoder, and why is it used only for reranking?
5. In Day 114, what did reranking the dense top 20 do to recall@1?
6. In Day 115, context recall@3 was 0.92 but accuracy 0.25. What does that tell you?
7. Write the symmetric quantisation formula for b bits.
8. Why did int4 per-tensor destroy SmolLM2-135M while int4 group-64 did not?
9. What is the core idea of GPTQ? Of AWQ?
10. Why was the Rust int8 kernel slower than torch fp32 at batch 1, and what did it gain?
11. What did the KV cache change in Day 118, and what does it cost?

12. Why was self-speculative decoding slower than plain decoding here?

Answers

1. It averages token vectors (masked) into one vector; normalising makes dot product equal cosine similarity.
2. Section chunking: recall@5 0.71 (vs 0.62-0.64 for word windows).
3. Score = Sum 1/(k + rank) over retrievers; it uses only ranks, so no score calibration is needed.
4. It attends across query and document jointly; it needs one forward per pair, so it is too slow for whole-corpus search (about 200 ms for 20 pairs here).
5. Raised it from 0.40 to 0.50 (recall@5 0.64 -> 0.81).
6. Retrieval mostly worked; the 135M generator failed to use the context: the main error is generation.
7. s = max|w| / (2^(b-1) - 1), q = clamp(round(w/s), -2^(b-1), 2^(b-1) - 1), ? = s.q.
8. Outliers (e.g. max 2.875 vs mean 0.1441 in one layer) set a huge single scale so most weights rounded to zero (ppl 1,561,367.93); group-64 scales adapt locally (ppl 36.89).
9. GPTQ: quantise columns one at a time and update the rest to compensate, using the Hessian X^TX. AWQ: scale up weight channels that meet large activations before quantising, folding the inverse scale into the input.
10. Torch uses optimised SIMD/blocked BLAS (15.9 µs vs our 145.2 µs with 8 threads); our kernel stored the weights in 4x less memory (884,736 B + 2,304 B scales vs 3,538,944 B).
11. Decode went from 27.3 to 67.8 tok/s with identical tokens; it costs memory growing linearly with length (94.4 MB at 2,048 tokens per sequence).
12. The early-exit draft was a poor predictor (acceptance 0.03-0.06), so draft steps were wasted and almost every token still needed a target forward.